从控制面可编程策略重新审视eBPF

The 4th eBPF Developer Conference (eBPFDC 2026)

任玉鑫

openEuler oEVP

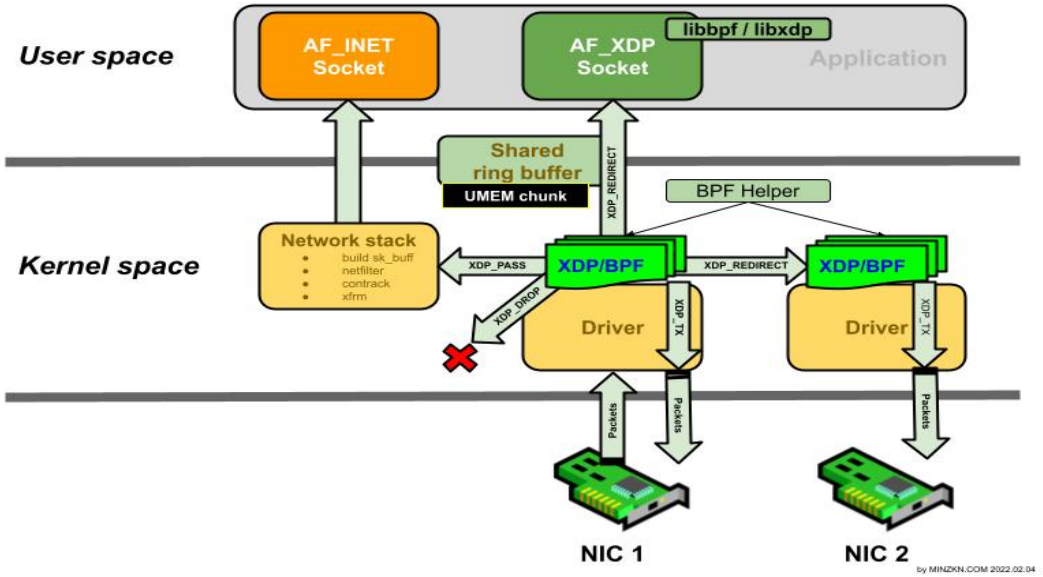
陕西 · 西安　　2026年4月18日

Data Plane Operation

```c
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>

struct {
    __uint(type, BPF_MAP_TYPE_XSKMAP);
    __uint(max_entries, 1);
    __type(key, __u32);
    __type(value, __u32);
} xsks_map SEC(".maps");

SEC("xdp")
int xdp_redirect_prog(struct xdp_md *ctx) {
    int rx_queue_index = ctx->rx_queue_index;
    if (bpf_map_lookup_elem(&xsks_map, &rx_queue_index))
    {
        return bpf_redirect_map(&xsks_map, rx_queue_index, 0);
    }
    return XDP_PASS;
}
```



Advantage

- in-wire processing
- in-kernel execution
- Short-cut acceleration
- Fine-grained customization

开放原子开源基金会 OPENATOM FOUNDATION | OpenEuler

Data Plane Policy

- Stateful policy algorithms
- Centralized decision-making with global network view
- Decide future behavior
- Complex invocation and scheduling timing
- Complicated update

开放原子开源基金会 OPENATOM FOUNDATION | OpenEuler

MaaS & Model Loading Bottleneck

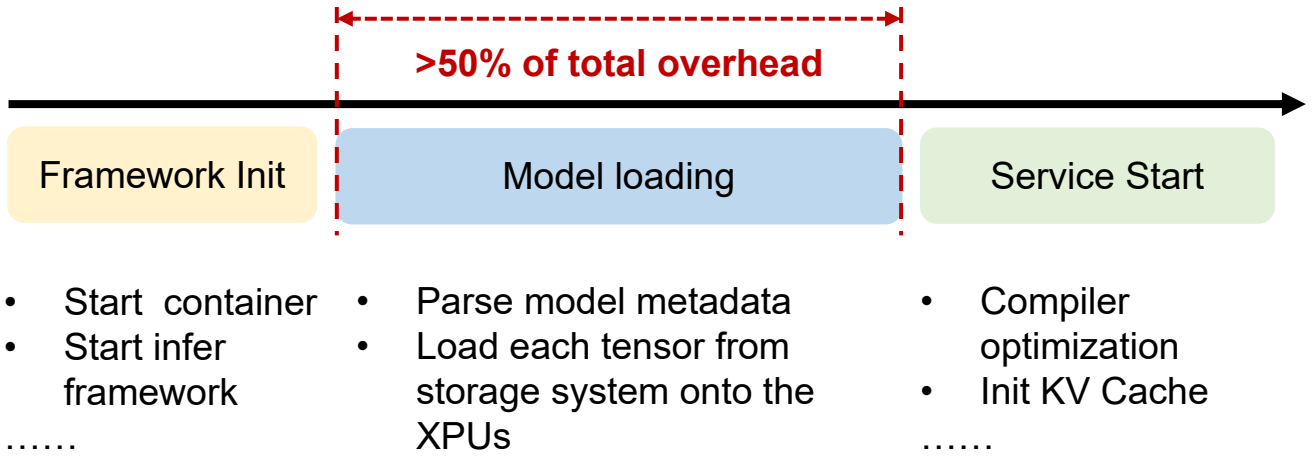
Deploying LLM inference services is a challenge for users

- Software complexity
- Hardware diversity
- High resource consumption

MaaS (Model-as-a-Service) Systems

- Run with one click
- Elastic deployment
- E.g., OpenAI, ModelArts, Bailian …

Process of LLM Inference Service Startup

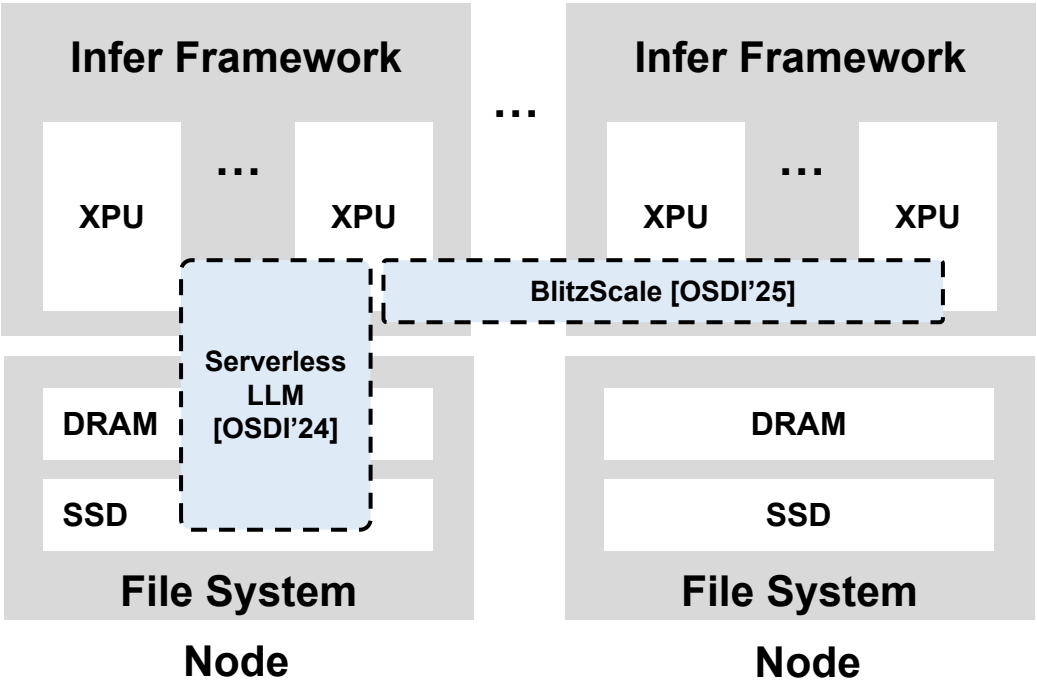


Model Loading is a main bottleneck in MaaS

- Inference services will be started frequently
- E.g., it takes about 1 hour to start *DeepSeek-R1-671B* from remote storage

Model Loading Optimizations

Incompatible Optimizations

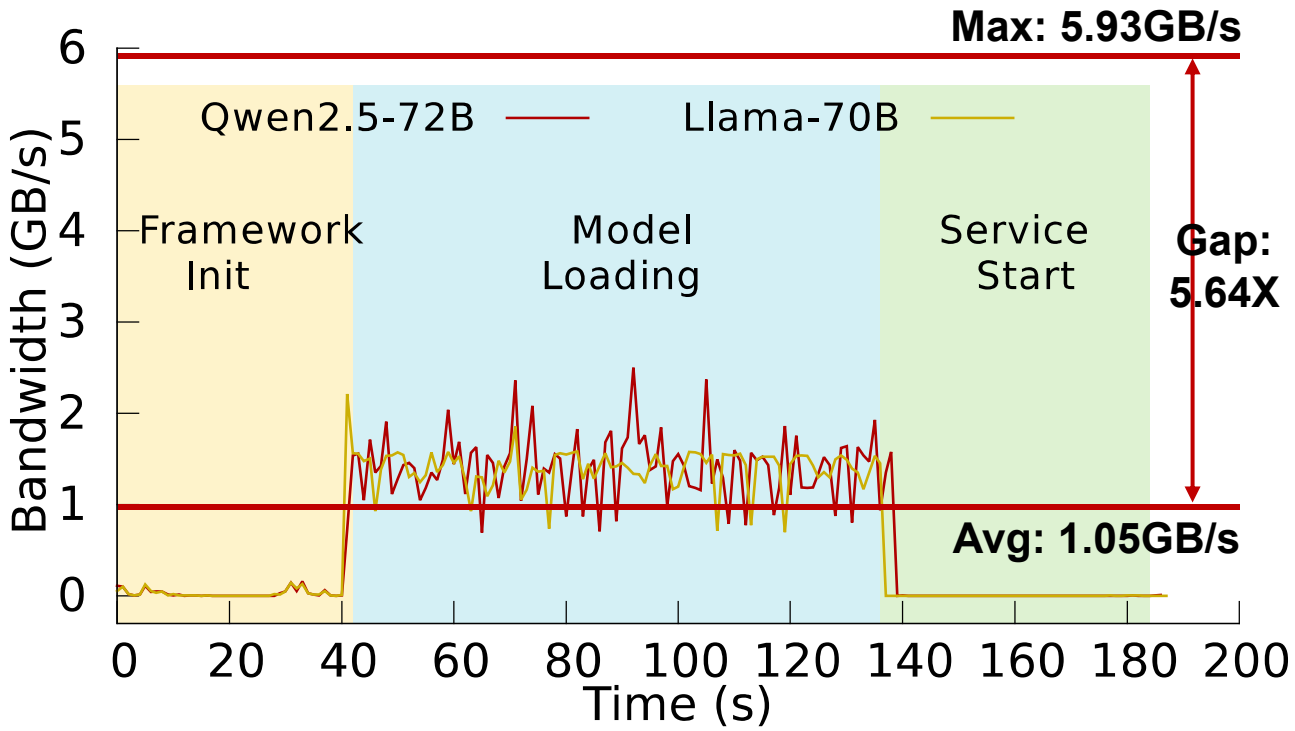


Compatibility is important in real-world scenarios

- Transparency in the LLM inference ecosystem
  - Software stacks are complex & diverse
  - Model format is difficult to change
  - It is a black box for infra providers
- No intrusive modifications to the OS kernel
  - Upstream is slow
  - Switching kernels is difficult
- No hardware dependency
  - Rich hardware architecture in datacenters

Goal: Achieve SOTA performance of model loading while ensuring compatibility

Analysis of FS Cache Policy in Model Loading



Experiment Setup: vLLM-Ascend 0.9.2, 4 NPUs,
Model @ SSD (EXT4), cold cache

Observation 1

- Prefetching cannot fully utilize idle SSD bandwidth and effectively hide I/O overhead

Observation 2

- XPU affinity is overlooked by prefetching, affecting the efficiency of host-to-device data transfer

Observation 3

- Eviction cannot precisely detect the I/O pattern of model loading, leading to cache thrashing

Motivation & Challenges

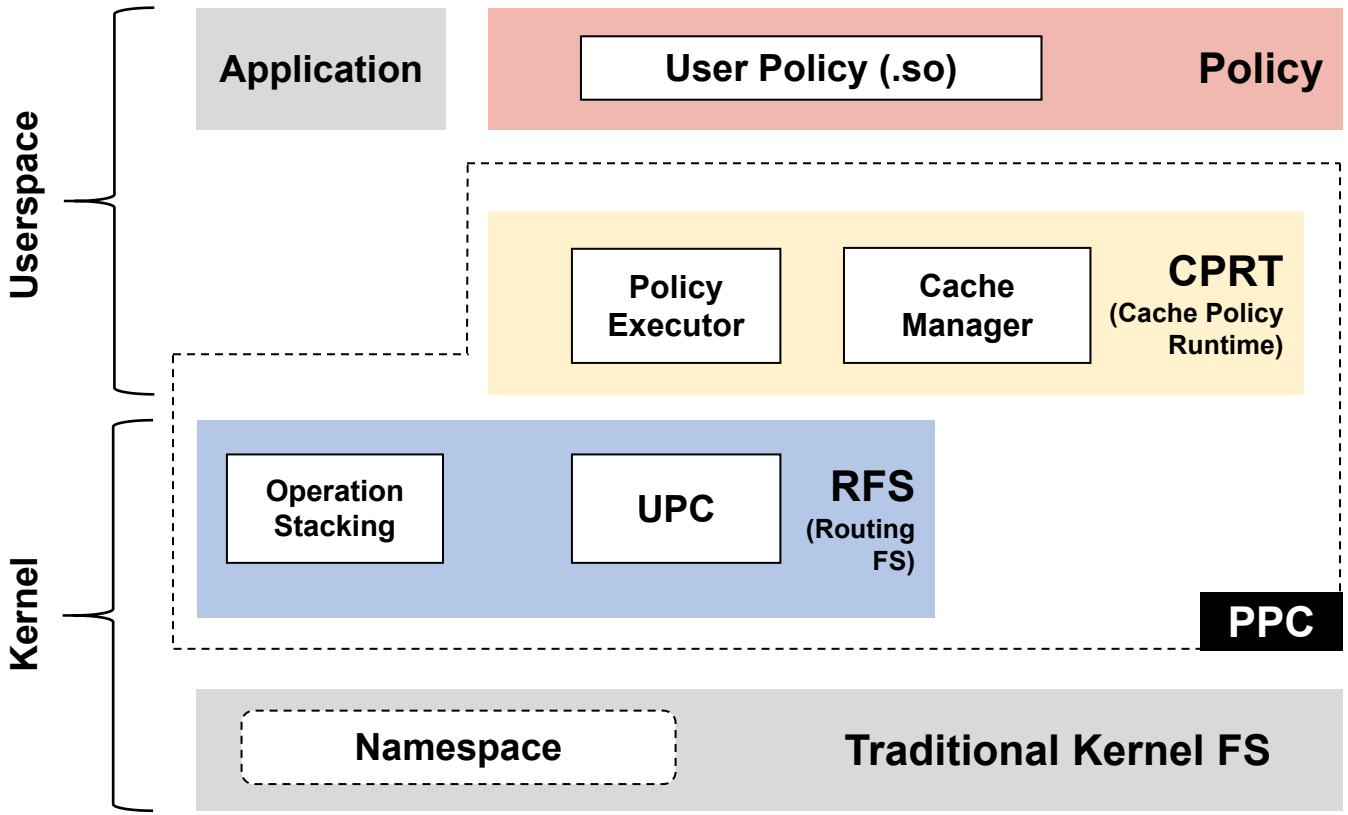
Make the kernel page cache programmable and enable the policy to adaptively optimize for different inference services

- **How to design an efficient programmable page cache framework?**
  - Non-intrusive
  - Flexibility
  - Lightweight

- **How to optimize the cache policy for different LLM inference services in a "black box" manner?**
  - I/O characteristics awareness
  - Adaptive I/O optimization

Programmable Page Cache (PPC)

Architecture of PPC



Routing File System (RFS)

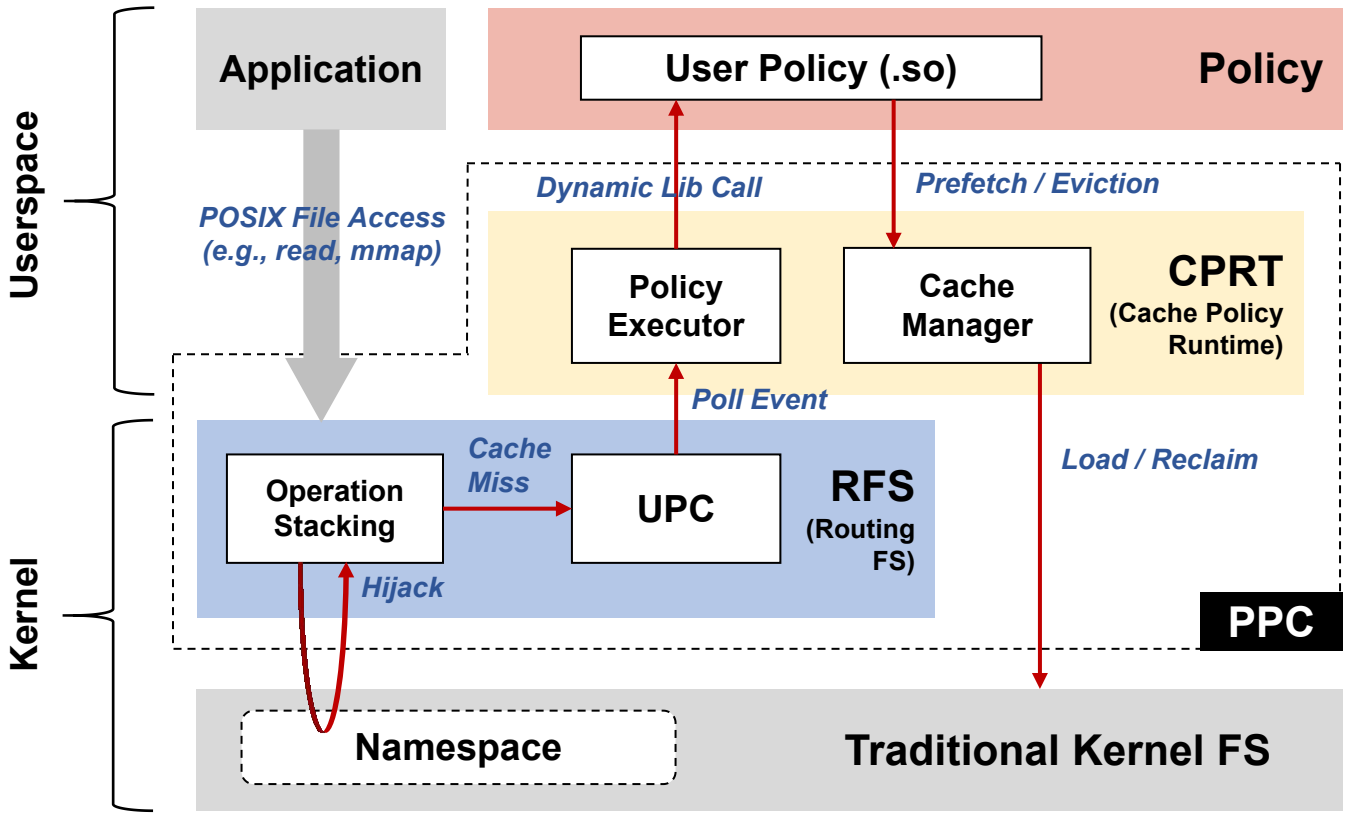
- **Read-only FS:** Mount on top of the existing kernel FS
- **Operations stacking:** Hijacks the operations of underlying FSes by overriding the VFS interfaces
- **UPC:** Redirect the cache policy logic to the user policy via userspace procedure call

Cache Policy Runtime (CRPT)

- **Policy Executor:** parses UPC events and provides VFS-like programming APIs
- **Cache Manager:** It is responsible for efficiently performing prefetching and eviction

Programmable Page Cache (PPC)

Architecture of PPC



Key Workflow

① Application uses the standard POSIX interface to access the file

② RFS hijacks I/O operations and checks for cache misses

③ (Cache miss) UPC encapsulates the message and triggers the user policy

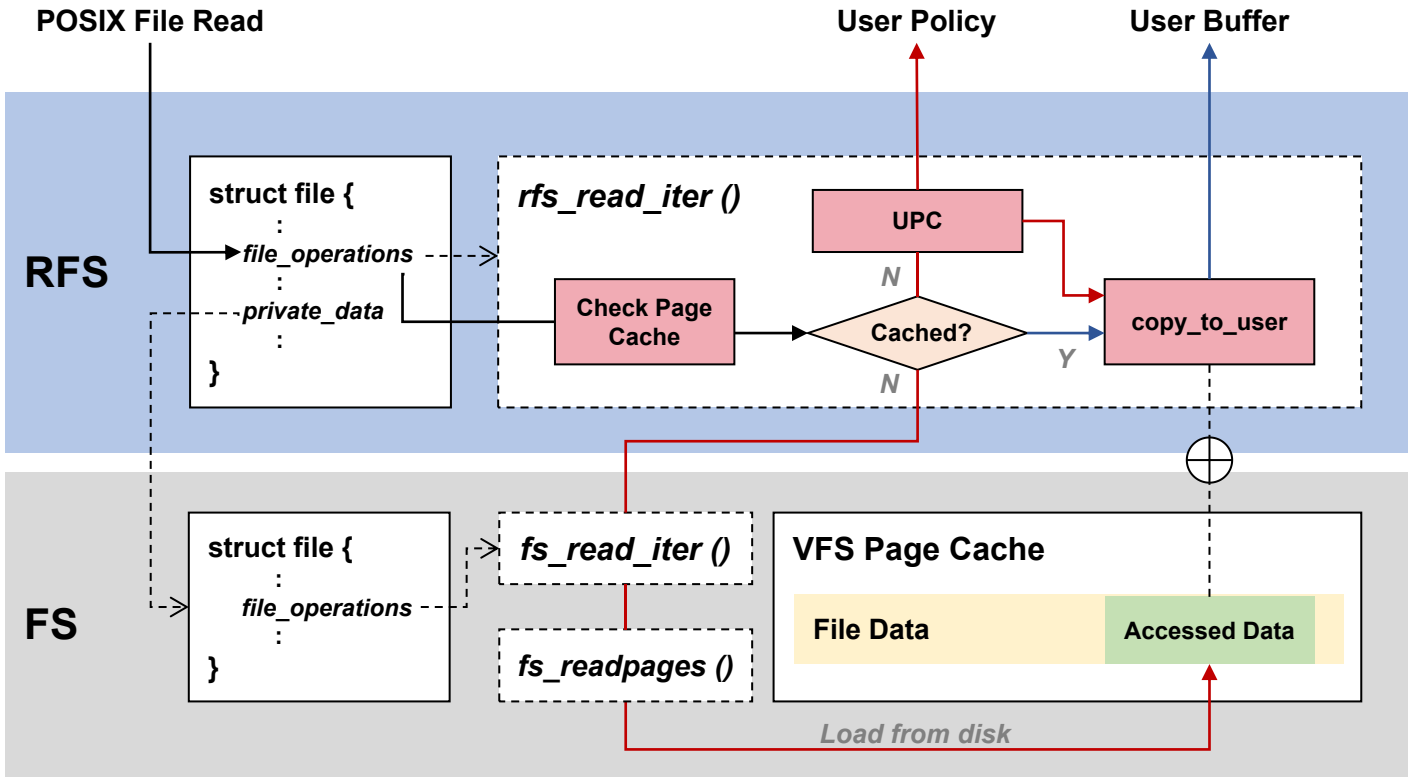
④ CPRT polls UPC events and converts them into dynamic library calls

⑤ Execute the user policy and return the set of data to be prefetched/evicted.

⑥ RTPC uses *ioctl/fadvise* to perform prefetching/eviction

Programmable Page Cache (PPC)

Routing File System (RFS)



Implementation

- Mount on top of the existing FS
- Core structures of RFS are mapped to the underlying FS
- Cache policies are embedded into the read and page fault processes
- Supports most read-only operations

Example: File Read

① Get file structure of RFS

② Call *read_iter* of RFS

③ (Cache hit) Read data from VFS page cache

④ (Cache miss) Redirect the read operation to underlying FS, invokes the user policy via UPC, and read data

Programmable Page Cache (PPC)

Cache Policy Runtime (CPRT)

init() & exit()

- Called once when the policy is registered/unloaded
- Used to initialize/destroy the runtime env. of the user policy

prefetch()

- Called every time an I/O miss event is received from UPC
- Utilize the miss I/O information to design prefetching policy
- Return a prefetch I/O list

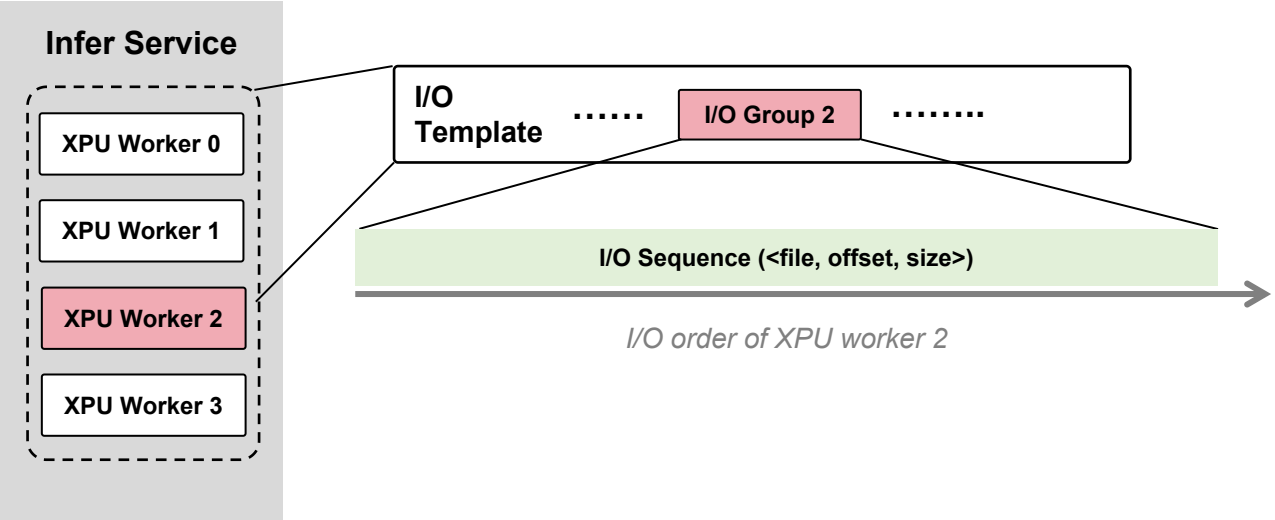
evict()

- Called when memory usage exceeds the threshold
- Utilize the miss I/O information to design eviction policy
- Return an eviction I/O list

```c
#include "ppc.h"
int init(void* private) {
        // Initialize the policy by "private"
        return 0;
}
int exit(void) {
        // Clean the environment and exit
        return 0;
}
struct prefetch_ios* prefetch(struct miss_io *mio) {
        // Utilize "mio" to design prefetch policy
        return ios_for_prefetch;
}
struct evict_ios* evict(struct miss_io *mio) {
        // Utilize "mio" to design eviction policy
        return ios_for_evict;
}
const struct ppc_operation policy = {
        .ppc_init = init,
        .ppc_exit = exit,
        .ppc_prefetch = prefetch,
        .ppc_evict = evict,
};
struct ppc_operation *register_policy() {
        return (struct ppc_operation *)&policy;
}
```

Case Study: Model-Accelerated I/O (MAIO)

I/O Template



Inspiration

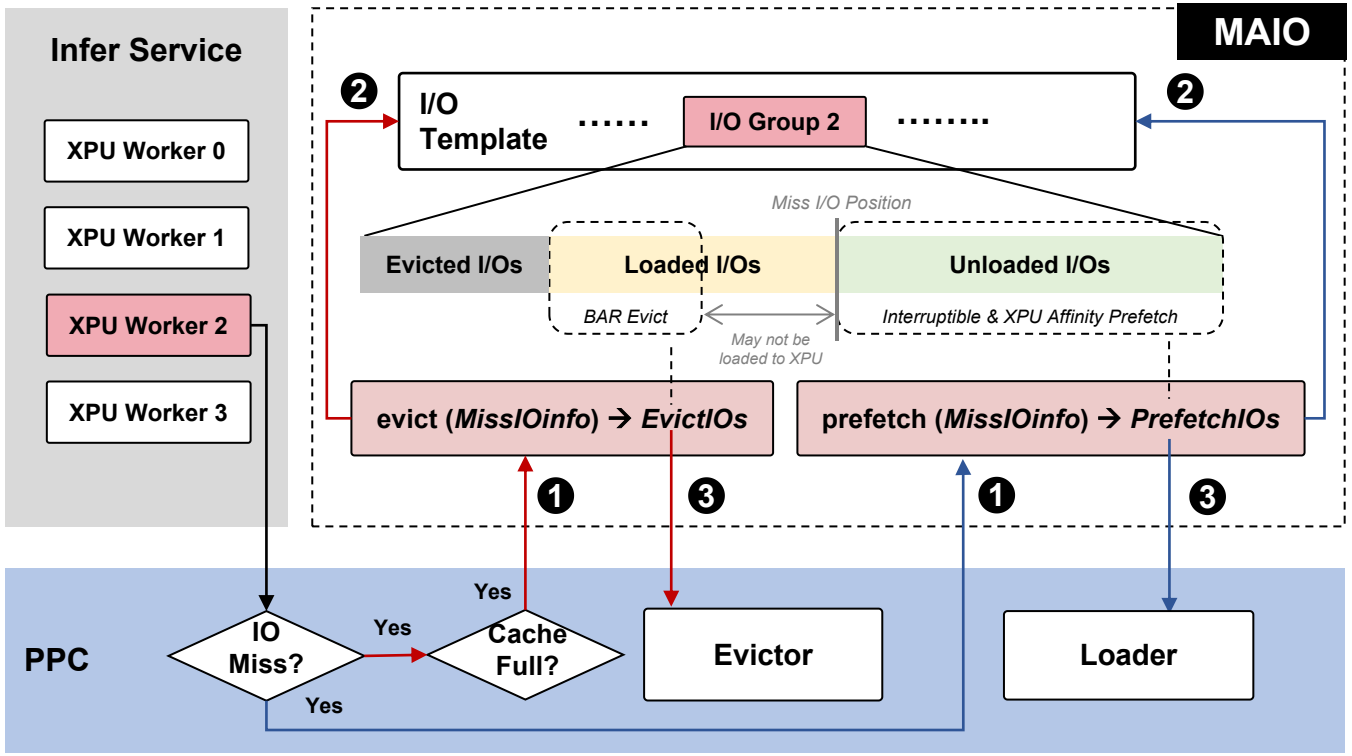
- The I/O pattern of model loading can be the same across identical LLM inference services
- Pre-building a template of the I/O pattern for each service

Organization

- Each XPU worker is associated with an I/O group
- Each group records the I/O sequence of the corresponding XPU worker during the model loading

Case Study: Model-Accelerated I/O (MAIO)

I/O Template-based Optimizations



Interruptible Prefetching

- A subsequent prefetch can interrupt an ongoing prefetch, which allows for aggressive prefetching windows

→ *Make full use of the idle bandwidth*

XPU Affinity Loading

- Dynamically detect the destination XPU of the data

→ *Accelerate H2D data transmission*

"Burn-after-Reading" Eviction

- Data already loaded into the XPU can be promptly evicted

→ *Avoid cache thrashing*

Evaluation Setup

Hardware

- 48-core Kunpeng 920 5250 CPU@2.60GHz * 4
- Ascend 910B2 NPU * 8
- 1TB DRAM + 3.75TB SSD

Software & Models

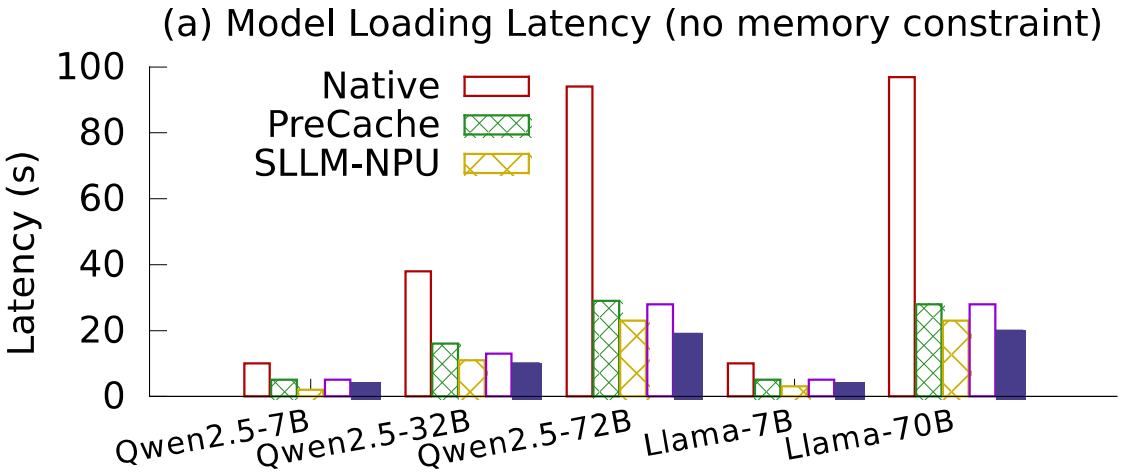
- vLLMAscend 0.9.2 + PyTorch 2.5.1 + Linux kernel 5.10
- Qwen2.5-7B, Qwen2.5-32B, Qwen2.5-72B, Llama-7B, Llama-70B

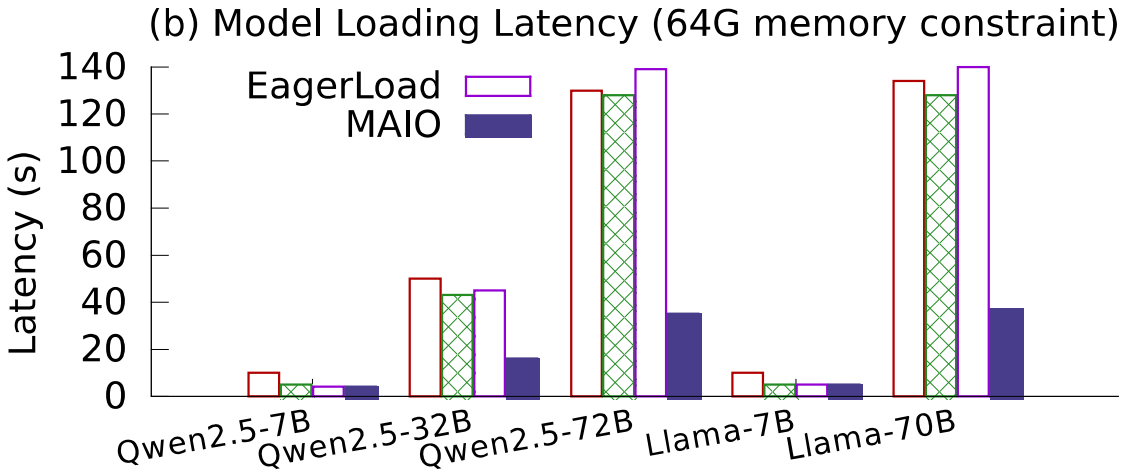
Tested Systems

- Compatible optimizations:
  - Native – native cache policy of the kernel file system
  - EagerLoad – a simple policy that prefetches the model into memory in one go
  - PreCache – pre-caches the entire model into memory (no disk I/Os)
  - MAIO — this work
- Incompatible optimizations:
  - ServerlessLLM-Ascend (SLLM-NPU) — reconstructing the model format and inference framework

Performance

Model Loading Latency



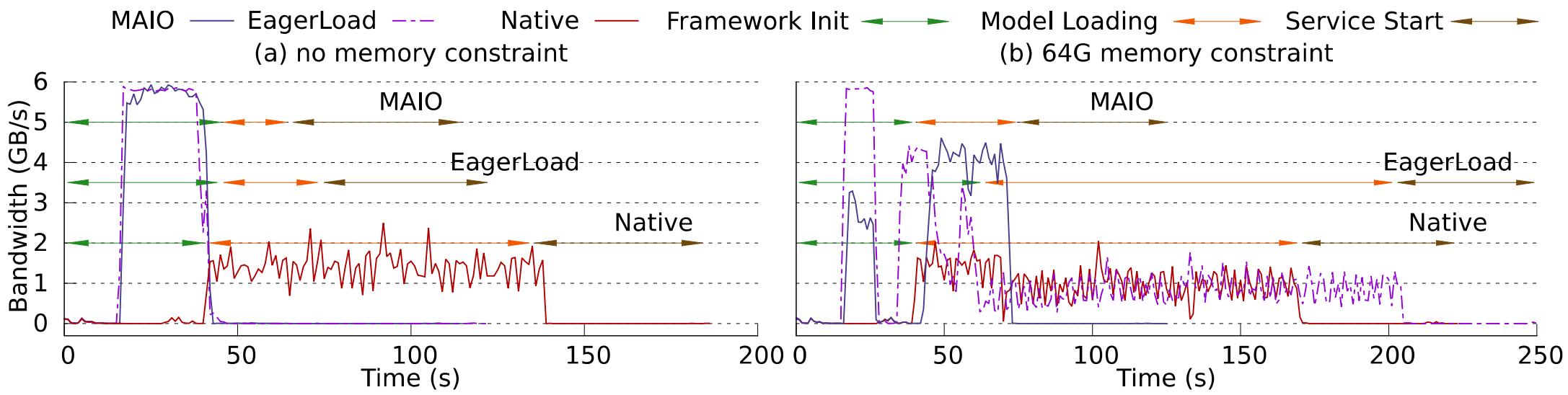


Summary

- **Sufficient memory:** up to 79% faster than other solutions; even better than the full cache solution (PreCache)
- **Limited memory (64GB):** up to 74% faster than other solutions

Performance

Bandwidth Utilization of E2E Startup

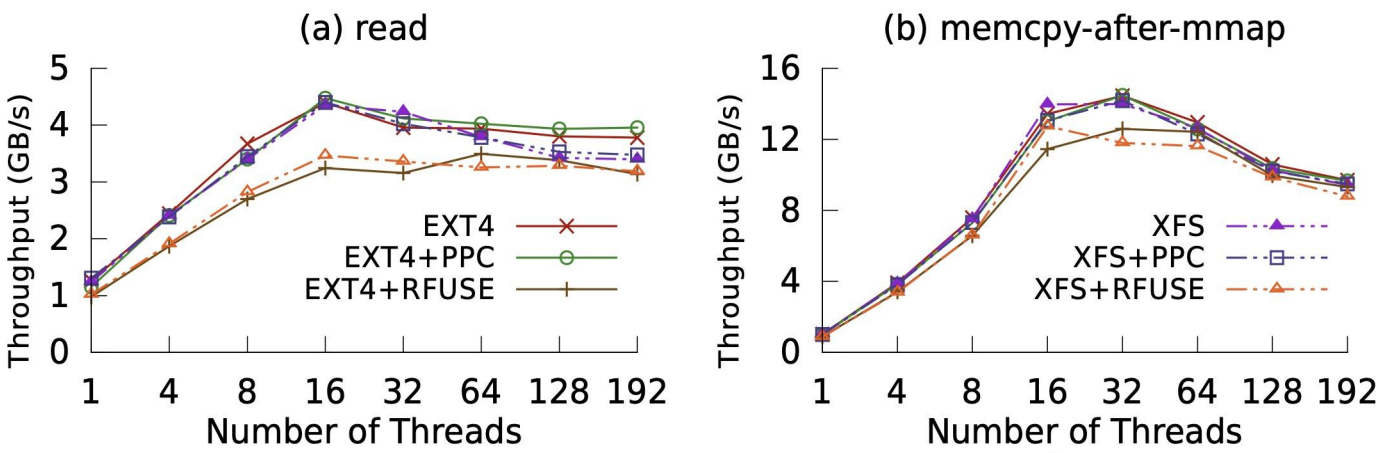


Summary

- **Sufficient memory:** I/Os of MAIO are concentrated during init. stage and fully utilize the maximum bandwidth
- **Limited memory (64GB):** MAIO precisely controls the use of cache space and offers the shortest E2E latency

Design Analysis

Overhead of PPC

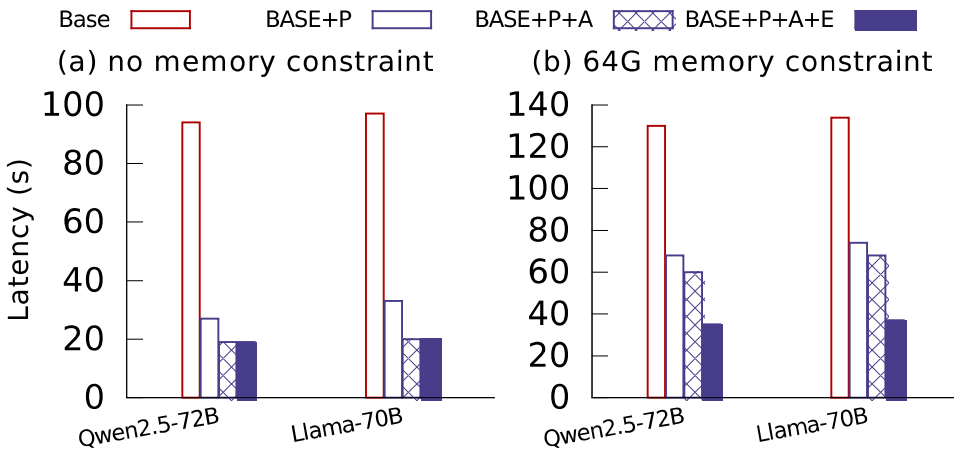


I/O Pattern: 1MB sequential I/Os on a 10GB file

Summary

- Additional overhead introduced by PPC is small (<3.7% @ EXT4)
- No impact on the scalability of the underlying FS

Performance Breakdown



+P: Interruptible prefetching; **+A:** XPU affinity loading;
+E: "Burn-after-Reading" eviction

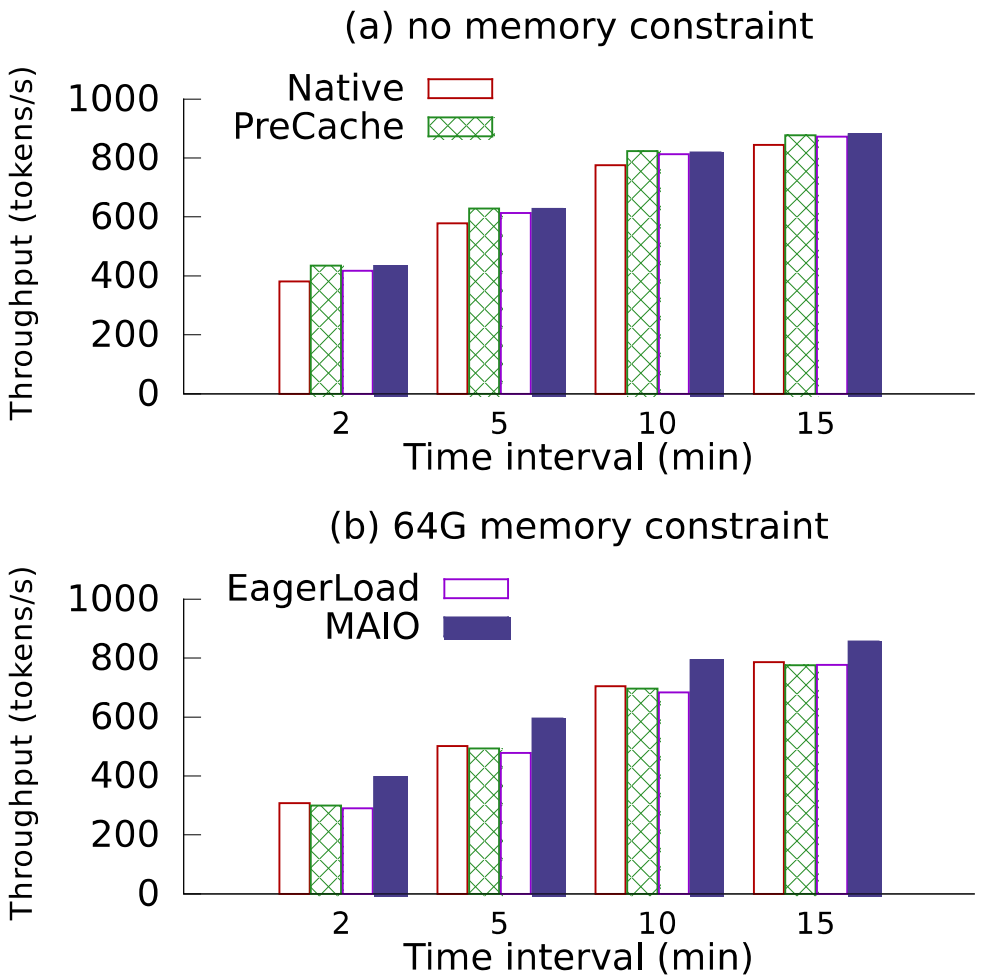
Summary

- Gradually enabling the P, A, and E brings about more than 44%, 4%, and 19% stacking improvements (@ limited memory)

Real-World Application

Elastic Deployment Workload

- Qwen2.5-72B, Llama-70B, and Qwen2.5-72B are run in sequence
- Inference service going through three stages: startup, inference execution, and then destruction
- ShareGPT is used as the input prompts



Summary

- MAIO delivers up to 13% and 28% compared to Native
- MAIO offers an average performance improvement of 19% and 21% in the limited memory scenario

Conclusion

- **AI Demands More Programmability**
  - Dynamic, fine-grained control plane policies programmability becomes critical
- **Code Generation by AI Agents**
  - Agents will write code. How to co-design agent and programmable framework?
- **eBPF ➡AI- eBPF**
  - eBPF is promising, but requires continuous evolution.

Source: *https://atomgit.com/openeuler/kernel/tree/OLK-6.6/fs/mfs [openEuler-24.03-LTS-SP3]*

Paper: *https://www.usenix.org/conference/fast26/presentation/liu-yubo [FAST'26]*

THANKS